

SYSTEM DESIGN INTERVIEW

Design a Stock Price Ticker

real-time market data fan-out to millions of clients (Robinhood / Yahoo Finance scale)

~ 40 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement
- 02** · Requirements
- 03** · Capacity Estimation
- 04** · High-Level Architecture
- 05** · Subscription & Fan-out Model
- 06** · Coalescing Algorithm
- 07** · Tick Message Format
- 08** · Transport: WebSocket vs Alternatives
- 09** · Time-Series Storage Path
- 10** · Latency Budget

- [11](#) · Failure Modes & Recovery
- [12](#) · Scaling the Broker Fleet
- [13](#) · Design Trade-offs
- [14](#) · Deep Dive: Hot Symbol Storms
- [15](#) · Interview Playbook

01

Problem Statement

Requirements and scope

Design a real-time **stock price ticker** like the Robinhood, Yahoo Finance, or TradingView watchlist. The system ingests raw exchange feeds (NYSE / NASDAQ over FIX or ITCH), normalises them, and pushes per-symbol price updates to **~10 million** concurrent clients with sub-100 ms p99 latency. Clients subscribe to small watchlists (10–50 symbols) and only need the latest price, not every tick.

Clarifying Questions

Question	Answer
Who are the clients?	Mobile and web users watching a personal watchlist (10–50 symbols)
Concurrent clients?	~10M during US market hours (9:30–16:00 ET)
Symbol universe?	~10K listed symbols (NYSE + NASDAQ common stock + ETFs)
Tick rate?	~5/sec/symbol average; ~500/sec on hot symbols (TSLA, AAPL on news)
Aggregate ingress?	~50,000 ticks/sec from exchange feed handler
Lossy or lossless?	Lossy — UI only needs latest tick; coalesce intermediate updates
Latency budget?	<100 ms p99: exchange tick → gateway → broker → client
History needed?	Intraday chart + 30 d / 1 y candles via separate TSDB path

INSIGHT

Why this problem is hard

Naïve fan-out is catastrophic. A hot symbol with 5M watchers at 100 ticks/sec would generate **500M push events/sec** — three orders of magnitude beyond what any broker fleet can serve. The whole design hinges on **per-client coalescing** plus careful subscription routing.

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Subscribe	Client opens WebSocket and sends SUBSCRIBE [AAPL, MSFT, ...]
Push	Server pushes price/size/time updates as ticks arrive
Coalesce	Throttle to ≤ 1 update/symbol/200 ms per client; drop intermediates
Snapshot	On subscribe, send last-known-price immediately from Redis
Unsubscribe	Client may modify watchlist mid-session without reconnect
History	Separate REST API for intraday and historical candles (TSDB)
Reconnect	Clients reconnect with last-seen seq; server replays gap or sends snapshot

Non-Functional Requirements

Requirement	Target
Concurrent clients	10M peak; 95% during market hours
Latency p50 / p99	20 ms / 100 ms (exchange → screen)
Availability	99.99% during market hours; market-hour SLO is sacred
Throughput (ingress)	50K ticks/sec normalised
Throughput (egress)	≤ 300 M push frames/sec post-coalesce across fleet
Loss tolerance	Lossy on intermediate ticks; never lose latest price
Data ordering	Per-symbol monotonic seq; cross-symbol unordered

03

Capacity Estimation

Math for scale

Ingress: Exchange Feed

- Universe: **~10K symbols** (NYSE + NASDAQ listed)
- Average tick rate: **5 ticks/sec/symbol** over the day
- Hot-symbol tick rate: **up to 500 ticks/sec** on news (TSLA, AAPL, NVDA)
- Aggregate ingress: **~50,000 ticks/sec** normalised
- Tick payload: ~40 B normalised (symbol_id, price, size, ts, seq) → **~2 MB/sec** ingress

Subscription Distribution

- Concurrent clients: **10M**
- Symbols per watchlist: **10–50**; assume avg **30**
- Total subscriptions: $10M \times 30 = \mathbf{300M}$ (client, symbol) pairs
- Average watchers per symbol: $300M / 10K = \mathbf{30,000}$ clients/symbol
- Hot symbol watchers: **5M** (TSLA at peak retail interest)

Egress: Naïve vs Coalesced

- Naïve hot path: $5M \text{ watchers} \times 100 \text{ ticks/sec} = \mathbf{500M}$ push events/sec (one symbol!)
- With **200 ms per-client coalesce**: ≤ 5 updates/sec/symbol/client
- Coalesced hot path: $5M \times 5/\text{sec} = \mathbf{25M}$ frames/sec for a hot symbol
- Fleet aggregate post-coalesce: **~10M clients** $\times$ **~30 active updates/sec** = 300M frames/sec across fleet
- Per broker (10K connections each): **~300K frames/sec/broker** — well within a single NIC

Storage (TSDB Path)

- Raw ticks: $50K/\text{sec} \times 86,400 \text{ sec} \approx \mathbf{4.3B}$ ticks/day
- Per tick on disk (compressed): ~10 B → **43 GB/day raw**
- Downsampled retention: 1 s for 24 h, 1 min for 30 d, 1 d forever
- Steady-state on-disk: **~1.5 TB/year** after compression

KEY

Key Numbers

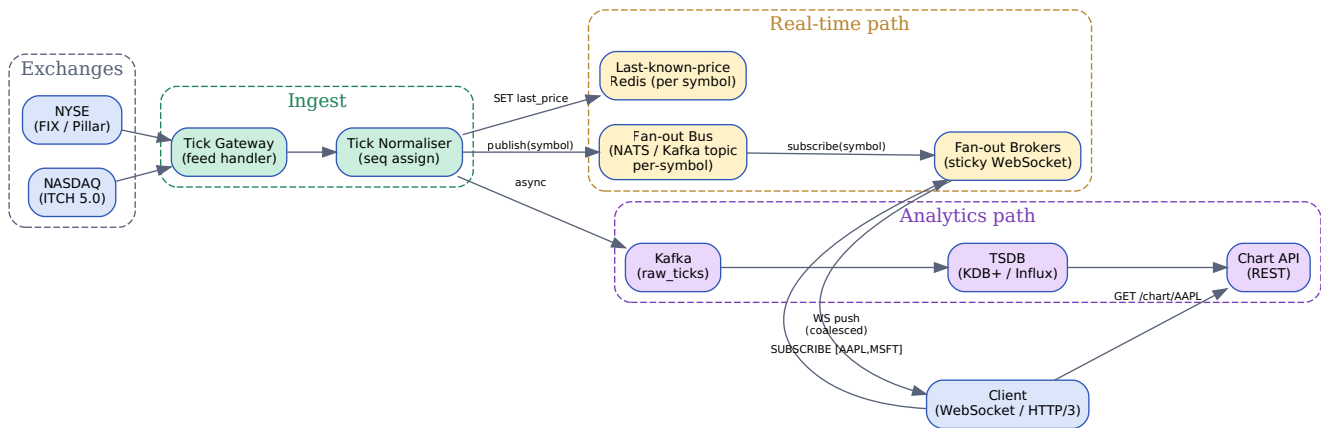
Ingress: **50K ticks/sec** · Concurrent clients: **10M** · Avg subs/client: **30** · Coalesce: **1/symbol/200 ms** · Latency budget: **100 ms p99**

04

High-Level Architecture

System overview

Two paths leave the **Tick Gateway**: a **real-time fan-out path** that pushes coalesced ticks to WebSocket brokers, and an **analytics path** through Kafka into the time-series database used for charts. Both fork from the **Tick Normaliser**, which converts raw FIX/ITCH frames into a uniform internal format and assigns a monotonic per-symbol seq.



Tick flow: exchange feeds ingest at the Tick Gateway, fork to a real-time path (Redis + fan-out brokers → WebSocket clients) and an analytics path (Kafka → TSDB → Chart API).

Component Responsibilities

- **Tick Gateway:** receives raw exchange frames; handles ITCH 5.0 / FIX session, sequencing, gap fills
- **Tick Normaliser:** maps exchange-specific message → uniform Tick struct; assigns monotonic seq per symbol
- **Last-known-price Redis:** hash keyed by symbol → latest price + seq; serves snapshots on subscribe
- **Fan-out Bus:** per-symbol topic (NATS subject or Kafka partition by symbol_id) — brokers pull only what their clients need
- **Fan-out Brokers:** stateful WebSocket servers; maintain symbol → client_set; perform per-client coalescing
- **Kafka raw_ticks:** durable lossless stream for the analytics / TSDB path
- **TSDB:** KDB+ or InfluxDB; powers chart API with downsampled bars

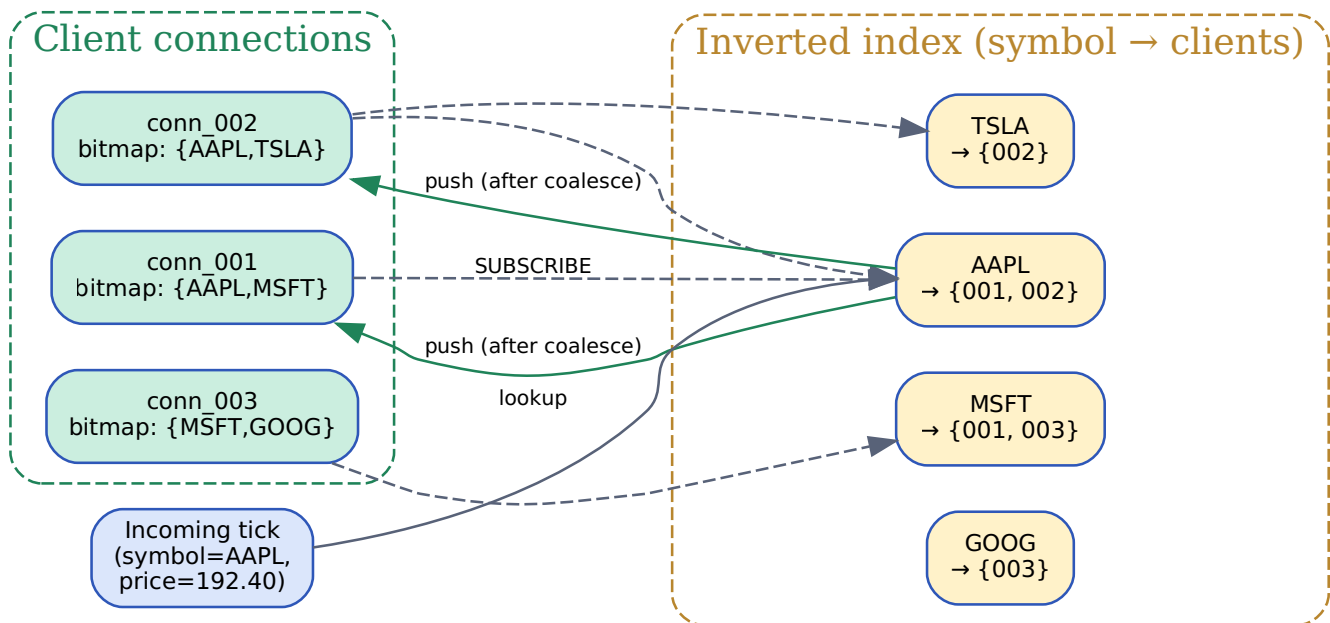
05

Subscription & Fan-out Model

Routing ticks to the right clients

The Mapping Problem

On every tick, a fan-out broker must answer: *which of my locally connected clients care about this symbol?* With ~10K connections per broker and 30 subs each, that's ~300K (client, symbol) edges per broker. We invert the map: symbol → client_set.



Per-broker subscription map: each client connection holds a sparse bitmap of subscribed symbols; broker also maintains the inverted symbol → clients index that drives fan-out on every tick.

Per-Connection State (in broker memory)

```
// One per connected client (resides in broker)
struct Connection {
    conn_id:      u64,
    user_id:      u64,
    socket:       WebSocket,
    // Sparse bitmap: symbol_id → bit. With 10K symbols a 1.25 KB
    // dense bitmap is fine; for 10-50 syms a small RoaringBitmap is
    // ~100 B and faster to iterate.
    subscriptions: RoaringBitmap,
    // Coalesce state (per subscribed symbol)
    last_pushed:  HashMap<symbol_id, (ts_ms, price)>,
    // Backpressure: bounded queue per connection
}
```



```

    out_queue:    RingBuffer<TickFrame, 256>,
    last_seq:     HashMap<symbol_id, u64>,
}

```

Broker-Wide Inverted Index

```

// Updated on every SUBSCRIBE / UNSUBSCRIBE
// Read on every incoming tick – must be lock-free or RW-locked
type SubIndex = ConcurrentHashMap<symbol_id, ConcurrentSet<conn_id>>;

fn on_tick(tick: Tick, idx: &SubIndex, conns: &ConnTable) {
    let Some(client_set) = idx.get(tick.symbol_id) else { return };
    for conn_id in client_set.iter() {
        let conn = conns.get(conn_id);
        if conn.should_emit(tick) { // 200 ms / 0.1% rule
            conn.enqueue(TickFrame::from(&tick));
        }
    }
}

```

Why a Fan-out Bus Per Symbol

- Each broker only needs the symbols its clients subscribe to — typically **~3K of 10K symbols** on a 10K-conn broker
- Use NATS subjects (`ticks.AAPL`, `ticks.MSFT`) or Kafka with partition-per-symbol
- Brokers join/leave subjects dynamically as clients sub/unsub: **no broker receives ticks for symbols nobody on it watches**
- Cuts inter-broker bandwidth from 50K msg/sec/broker to ~15K msg/sec/broker

06

Coalescing Algorithm

Lossy throttling that preserves the latest price

A ticker UI updates a number on a screen — the user can't perceive more than ~5 changes/sec. We exploit this: instead of pushing every tick, push at most **one update per symbol per 200 ms per client**, but always honour a **significant price move ($\geq 0.1\%$)** immediately. This is *conflated* data — a standard pattern in market-data distribution.

Per-Client Coalesce Rule

```
const COALESCE_MS:    u64 = 200;    // throttle window
const PRICE_DELTA_BP: f64 = 10.0;   // 10 bps = 0.1%

fn should_emit(conn: &Connection, tick: &Tick) -> bool {
    let now = now_ms();
    match conn.last_pushed.get(&tick.symbol_id) {
        None => true,                    // first tick → push
        Some((last_ts, last_px)) => {
            let age_ok = now - last_ts >= COALESCE_MS;
            let move_bp = ((tick.price - last_px).abs() / last_px)
                          * 10_000.0;
            let big_move = move_bp >= PRICE_DELTA_BP;
            age_ok || big_move
        }
    }
}

fn on_emit(conn: &mut Connection, tick: &Tick) {
    conn.last_pushed.insert(tick.symbol_id, (now_ms(), tick.price));
    // Buffer is a fixed ring; if full → drop oldest non-latest entry
    conn.out_queue.push_or_replace(tick.symbol_id, TickFrame::from(tick));
}
```

Why This Works

- **Bounded egress:** ≤ 5 updates/sec/symbol/client $\rightarrow 30 \text{ syms} \times 5 = 150 \text{ frames/sec/client max}$
- **Latest-wins semantics:** ring buffer holds at most one entry per symbol — overwrite, don't append
- **Big moves bypass throttle:** a 0.1% jump on AAPL pushes immediately even within the 200 ms window
- **No global lock:** coalesce state lives on the connection — a single thread per shard owns it

Coalesce vs No-Coalesce Trade-off

Mode	Egress	Latency	Use Case
No coalesce (raw)	500M frames/sec hot symbol	Best — 5–10 ms	HFT / pro traders paying for L2 feed
Coalesce 200 ms	~5/sec/sym/client	20–100 ms (bounded)	Retail ticker (Robinhood, Yahoo)
Coalesce 1 s	~1/sec/sym/client	Up to 1 s	Cheap watchlist, free tier

TIP

Per-tier coalesce windows

Make the 200 ms window a per-tier knob. Free users get 1 s, premium 200 ms, professional 50 ms. Same broker code, just a config field per Connection — saves ~5× egress on the long tail.

07

Tick Message Format

Wire protocol on the WebSocket

Goals

- **Compact:** ticker frames go over mobile data — every byte counts at 150 frames/sec
- **Self-describing seq:** client must detect gaps and request snapshot
- **Multiplexed:** one WebSocket carries many symbols
- **Binary:** Protobuf or FlatBuffers — JSON adds 3–5× overhead

Protobuf Schema

```
syntax = "proto3";

// Server → client
message TickFrame {
    uint32 symbol_id = 1;    // dictionary lookup; 4 B vs 5-6 B string
    sint64 price_e6 = 2;    // price × 1e6 as int (avoid float)
    uint32 size = 3;        // last trade size
    fixed64 ts_us = 4;      // exchange timestamp, microseconds
    uint64 seq = 5;         // monotonic per symbol
    uint32 flags = 6;       // bid/ask/trade/halt bits
}

// Client → server
message ClientMsg {
    oneof body {
        Subscribe subscribe = 1;
        Unsubscribe unsubscribe = 2;
        Resync resync = 3;
    }
}

message Subscribe { repeated uint32 symbol_ids = 1; }
message Unsubscribe { repeated uint32 symbol_ids = 1; }
message Resync { uint32 symbol_id = 1; uint64 last_seen_seq = 2; }

// Multiplexed envelope (one per WS frame)
message ServerMsg {
    repeated TickFrame ticks = 1; // batched per flush
    Snapshot snapshot = 2; // sent on subscribe / resync
    uint32 error = 3;
}
```

```
message Snapshot { uint32 symbol_id = 1; sint64 price_e6 = 2;
                  uint64 seq = 3; fixed64 ts_us = 4; }
```

Symbol Dictionary

- Client downloads `symbol_id` ↔ ticker dictionary on app start (≈ 200 KB, gzipped, cached)
- Refreshed once per trading day (new IPOs, delistings)
- Lets us send **4-byte symbol_id** instead of 5–6 B ASCII tickers

Wire Size

- Per TickFrame on the wire: **~22–28 B** (varint-packed)
- Batch 8 ticks per WS frame → **~200 B** + 4 B WS header
- $10\text{M clients} \times 30 \text{ active syms} \times 5 \text{ Hz} \times 25 \text{ B} \div 8 = \textbf{\sim 470 MB/sec}$ outbound across the fleet

08

Transport: WebSocket vs Alternatives

Why long-lived push, with HTTP/3 fallback

Comparison

Transport	Push?	Pros	Cons
WebSocket	Yes (full duplex)	Binary; low overhead; widely supported; works through proxies	Sticky session; one TCP per client; can stall under packet loss
SSE (Server-Sent Events)	Server → client only	Plain HTTP; auto-reconnect; firewall-friendly	Text-only; no client → server multiplex; one-way subscribe is awkward
HTTP/3 (QUIC) streams	Yes (per-stream)	No head-of-line blocking; better on lossy mobile	Proxy support uneven; library maturity
Long-poll	Simulated	Works anywhere	30K reqs/sec on 10M clients = unbearable; high latency

Recommendation

- **Primary:** WebSocket over TLS 1.3, HTTP/2 upgrade
- **Fallback:** HTTP/3 long-poll for clients behind corporate proxies that block WS
- **Heartbeat:** binary PING every 25 s; client ACK or reconnect after 60 s silence
- **Compression:** permessage-deflate *off* — Protobuf is already compact, deflate adds CPU

Sticky Sessions

Each WebSocket terminates on a single broker that owns its subscription state. The L7 load balancer (Envoy / Nginx) hashes by `conn_id` cookie or by client IP for the initial upgrade request. After that, the WS is pinned for the session — moving it would require state migration.

WARNING

Sticky vs stateless

Sticky sessions simplify state but couple availability to broker uptime. A broker crash drops 10K connections. Mitigate with: fast client reconnect (≤ 2 s), **last-seen seq replay**, and **graceful drain** for deploys (50% drain over 30 s).

09

Time-Series Storage Path

Charts, candles, history

Why a Separate Path

The real-time path is lossy and only carries the latest price. The TSDB path is **lossless** — every tick is durably written to Kafka, then ingested into a time-series database that powers chart APIs (1-minute candles, intraday history, daily bars).

TSDB Comparison

Database	Strengths	Weaknesses	Use
KDB+	Industry std for tick data; columnar; q-language analytics	Expensive license; niche skills	Hedge funds; HFT-grade tick storage
InfluxDB	Easy to operate; good downsampling (continuous queries); HTTP API	Cardinality limits; OSS scaling pain	Mid-scale ticker, dashboards
TimescaleDB	Postgres compatibility; SQL; hypertables	Write throughput vs purpose-built TSDBs	If you already run Postgres
Cassandra	Massive write throughput; multi-region	Range scans across symbol+time slow; manual aggregation	Raw archive only

Retention & Downsampling

Granularity	Retention	Bars/day/symbol	Use
1-second	24 hours	~23,400	Intraday high-detail chart
1-minute	30 days	390	Default chart view
1-day OHLCV	Forever	1	Long-term history; analytics

Ingestion

- Tick Normaliser publishes to Kafka topic `raw_ticks` (partition by `symbol_id`, RF=3)
- TSDB writer consumes; batches 1000 ticks per flush; **at-least-once** semantics

- Downsampler runs as Kafka Streams or Flink job: tick → 1 s bar → 1 min bar → 1 d bar
- Continuous queries (Influx) or scheduled kdb+ tick rollups (KDB+)

10

Latency Budget

100 ms p99 — where does it go?

Hop-by-hop

Hop	Budget	Notes
Exchange → Tick Gateway	1–3 ms	Co-located in NYSE / NASDAQ data centres; cross-connect
Gateway → Normaliser	< 1 ms	Same host or shared-memory ring buffer
Normaliser → Fan-out Bus	1–5 ms	NATS in-memory pub/sub; one DC hop
Bus → Broker	1–3 ms	Brokers subscribe to needed subjects; in-region
Broker coalesce buffer	0–200 ms	By design: throttling window; bounded
Broker → client (network)	20–60 ms	Mobile RTT dominates; TLS established already
Total p99	< 100 ms	Budget mostly absorbed by mobile last-mile + coalesce

Where Things Go Wrong

- **GC pauses on broker:** use Rust / Go with low-pause GC; budget <5 ms p99
- **NIC saturation:** 100 Gbps NICs; monitor tx_drop
- **Backpressure on client:** slow mobile fills socket buffer → ring drops oldest non-latest
- **Cross-AZ hop:** keep gateway, bus, brokers *in the same AZ* as the exchange feed

INSIGHT

Co-location matters

Even a single trans-continental hop (~70 ms one-way US East ↔ West) blows the 100 ms budget. Run a full ingest+fan-out stack *per region* with replicated Kafka for the analytics path only. Real-time path is regional, not global.

11

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
Exchange feed gap (seq jump)	Missing ticks; stale prices	Sequence-gap monitor in Normaliser	Backfill from secondary feed (e.g., NASDAQ TotalView ↔ NYSE OpenBook); request retransmit; mark symbol stale until repaired
Primary feed disconnect	All ticks halted	Heartbeat from FIX session; 5 s timeout	Auto-failover to redundant feed; alert; clients keep last-known-price with 'stale' badge
Fan-out broker crash	10K connections drop	Health check; LB ejects	Clients reconnect with <code>resync(symbol, last_seen_seq)</code> ; broker fleet auto-scales
Redis last-price down	New subscribers see no snapshot	Redis sentinel alert	Failover to replica; brokers serve stale snapshot from local cache for 30 s
Kafka raw_ticks lag	TSDB stale; charts behind	Consumer-lag metric	Real-time path unaffected; scale TSDB writers; replay from Kafka
Hot-symbol overload	Broker CPU spike on news	Per-symbol rate metric	Per-symbol surge protection: drop coalesce window to 500 ms for that symbol fleet-wide
Network partition	Cross-AZ unreachable	Connection error rate	Each AZ has its own ingest+fan-out; client reconnects to nearest healthy broker via DNS

Client Reconnect Protocol

1. Client detects WS close or 60 s silence
2. Reconnects with exponential backoff capped at 5 s; resolves DNS to nearest broker
3. Sends Subscribe + per-symbol `last_seen_seq` (resync messages)
4. Broker compares against Redis last-price + Kafka tail; sends Snapshot if gap is non-trivial
5. Client merges snapshot, resumes streaming

WARNING

Market-hour SLO is sacred

9:30 am ET (US market open) is the worst time for an outage. Freeze deploys 30 min before open through close. Use canary brokers (1% of fleet) with auto-rollback if error rate > 0.5%.

12

Scaling the Broker Fleet

From 1M to 10M concurrent connections

Broker Capacity Planning

- Per broker: **10K connections** (limited by ulimit, FDs, kernel buffers)
- Memory: $10K \times \sim 30 \text{ KB/conn}$ (sockets + bitmap + coalesce state) = **~300 MB/broker**
- CPU: $\sim 300K$ frame writes/sec; on 16-core m6i.4xlarge: $\sim 30\%$ CPU at peak
- Fleet size: $10M / 10K = \mathbf{1,000 \text{ brokers}}$; over-provision to 1,500 for headroom + canaries

Auto-scaling Triggers

- **Scale up:** $>7K$ connections per broker for 5 min
- **Scale up:** p99 broker CPU $>70\%$ for 5 min
- **Scale down:** $<3K$ connections per broker for 30 min (sticky drains take time)
- **Pre-warm:** 30 min before US open, scale fleet to 120% of yesterday's peak

Sharding the Subscription Index

The inverted symbol $\rightarrow$ client_set index is *per-broker* — not global. Each broker only knows its own 10K clients. Scaling brokers does not scale a shared structure; the fan-out bus does the global routing. This is what makes **linear horizontal scaling** possible.

Dictionary / Symbol Updates

- New listings (IPOs) added overnight; brokers reload dictionary at session boundary
- Delistings $\rightarrow$ mark symbol_id as *removed* for 30 days; reject new subs
- Corporate actions (splits, dividends) emitted as control frames; client adjusts last_pushed cache

13

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Coalesce ticks?	Yes (200 ms / 0.1%)	Bounds egress at 10M-client scale; loses intermediate prints. Acceptable for retail UI; <i>not</i> for HFT.
Loss model	Lossy real-time, lossless analytics	Two paths add complexity; preserves history while keeping live cheap.
Transport	WebSocket primary, HTTP/3 fallback	Sticky sessions complicate failover but give 30% lower CPU than long-poll at 10M clients.
Sticky vs stateless	Sticky	Crashes drop 10K conns at once; clients reconnect in <2 s. Stateless would need a global sub index — much costlier.
Bus	Per-symbol topic (NATS / Kafka)	Avoids broadcasting all 50K ticks/sec to every broker; adds many topics.
Snapshot source	Redis last-known-price	Adds a hot-path Redis dep; saves replaying Kafka on every subscribe.
TSDB	KDB+ (or Influx)	Best-in-class for ticks; license cost. Cassandra alone would need huge custom rollup work.
Region model	Real-time per-region; analytics global	Latency budget forces it; a NYSE-listed symbol still has ~70 ms RTT to APAC clients no matter what.
Symbol IDs	Numeric dictionary, not strings	Saves ~30% wire bytes; clients must keep dictionary in sync.
Compression	Off (rely on Protobuf)	Lower CPU on broker; loses ~10% bandwidth — net win on a 10M-conn fleet.

14

Deep Dive: Hot Symbol Storms

The TSLA-on-news problem

On news days, a single symbol can spike from 5 ticks/sec to **500 ticks/sec** with millions of watchers. Even after coalesce, every broker that has TSLA subscribers must process 500 incoming ticks/sec for that symbol, fan out to ~5,000 local subscribers each, and apply the per-client coalesce check. This is the *single biggest CPU spike* in the system.

Mitigations

- **Hot-symbol coalesce at the bus:** fan-out bus pre-coalesces to 50 Hz before delivering to brokers — cuts broker work by 10×
- **Adaptive window:** when broker CPU > 70%, dynamically widen coalesce from 200 ms to 500 ms *for that symbol only*
- **Frame batching:** brokers flush WS write queue every 50 ms — combine multiple symbols into one frame, amortising syscall cost
- **Drop-oldest ring buffer:** if a slow client can't keep up, drop intermediates — never block the broker
- **Per-symbol broker pinning:** for top-100 symbols, dedicate a broker shard with extra cores

Adaptive Coalesce Pseudocode

```
// Runs once per second per broker
fn adapt_coalesce(broker: &mut Broker) {
    let cpu = read_cpu_pct();
    let window = match cpu {
        c if c > 85 => 1000,    // emergency: 1 s
        c if c > 70 => 500,    // throttle
        c if c > 50 => 300,
        _            => 200,    // default
    };
    for sym in broker.hot_symbols() { // top-N by tick rate
        broker.coalesce_window.insert(sym, window);
    }
    // Long-tail symbols stay at 200 ms regardless
}
```

TIP

Halt frames bypass everything

Trading halts (LULD, news pending) must reach clients in <1 s — they are *control* messages, not price ticks. Mark them with a flag bit; brokers bypass coalesce and emit immediately.

15

Interview Playbook

How to present this

This is a fan-out problem dressed up as a market-data problem. The interview signal is your ability to spot the **multiplicative blow-up** ($5M \times 100 = 500M/sec$) and design coalescing as the first-order defence. Numbers must be self-consistent.

45-Minute Interview Outline

1. **Intro (2 min)**: clarify scale (10M clients, 30 syms, 50K ticks/sec, 100 ms p99)
2. **Capacity (5 min)**: compute the 500M/sec naïve fan-out — make the interviewer sweat
3. **High-level arch (5 min)**: two paths (real-time vs analytics); fan-out bus per symbol
4. **Subscription model (8 min)**: per-broker inverted index; sticky WebSocket; bitmap per conn
5. **Coalesce (8 min)**: 200 ms / 0.1% rule; ring buffer with overwrite; per-tier windows
6. **Wire format (5 min)**: Protobuf, symbol_id dictionary, batching
7. **Failures (5 min)**: feed gap → backfill; broker crash → resync(seq); hot symbol → adaptive window
8. **Trade-offs (4 min)**: sticky vs stateless, lossy vs lossless, real-time vs analytics paths
9. **Wrap (3 min)**: repeat memorize-these-numbers; offer extension (options, L2 book)

Key Talking Points

- “Naïve fan-out for one hot symbol = 500M events/sec — coalescing is non-negotiable”
- “Per-symbol fan-out bus means each broker only pulls the ~3K symbols its clients want”
- “200 ms / 0.1% coalesce: bounded egress and bounded staleness simultaneously”
- “Real-time path is lossy by design; lossless TSDB path is the system of record”
- “Sticky WebSocket trades crash blast radius (10K conns) for simpler state”
- “Co-locate gateway, bus, brokers in same AZ as exchange feed — physics, not engineering”

Common Follow-ups & Answers

- **Q: How do you handle a client subscribed to TSLA when the broker crashes?** A: WS closes, client reconnects in <2 s, sends `resync(TSLA, last_seen_seq)`; new broker fetches snapshot from Redis (last-known-price) and resumes streaming.
- **Q: Why not just use a shared Redis pub/sub?** A: Redis pub/sub is broadcast — every subscriber gets every message on the channel. Per-symbol fan-out at scale needs broker-side filtering anyway, plus Redis pub/sub is lossy on disconnect. NATS or Kafka with per-symbol topics + durability for replay is the right tool.

- **Q: How do you bound memory on a slow client?** A: Bounded ring buffer per connection (256 frames). Slow client → ring fills → we overwrite the oldest *non-latest* entry per symbol. Latest price always survives; intermediates are sacrificed.
- **Q: Do you need ordering?** A: Per-symbol monotonic seq, yes — derived from exchange seq. Cross-symbol ordering is irrelevant (UI doesn't care if MSFT or AAPL updates first).
- **Q: What about pre-market / after-hours?** A: Tick rate is ~10× lower, so capacity is fine. Same code path; flag in tick payload distinguishes session.
- **Q: How do you bill / meter this?** A: Each subscribe goes through an entitlement check (tier → max symbols, coalesce window). Audit via Kafka entitlements topic.

KEY

Memorize These Numbers

50K ticks/sec ingress · **10K symbols** · **10M clients** · **30 subs/client** avg · **30K watchers/symbol** avg, **5M** hot · **200 ms / 0.1%** coalesce · **100 ms p99** end-to-end · **10K conns/broker**, **~1,000 brokers** in fleet.